Lab 2：BombLab 实验报告

1 实验目标与方法

本实验通过 GDB 分析个人 `bomb`，恢复六个正式关卡与 Honor 的校验关系，并在 `defuser.c` 中实现根据 challenge 计算口令的函数。

主要使用以下命令查看汇编：

```
gdb ./bomb
set pagination off
disassemble phase_01
```

记 64 位无符号 challenge 为 C，$\mathrm{u32}(z)$ 表示取低 32 位，$\oplus$ 表示按位异或。下文 32 位加减乘均按模 2^{32} 运算；对 challenge 的移位按相应 64 位指令处理，32 位中间量的移位保持 32 位语义。整数参数依次通过 `rdi,rsi,rdx,rcx,r8,r9` 传入，32 位结果由 `eax` 返回。

2 各关卡推演过程

2.1 Phase 1：函数调用

`phase_01` 要求输入一个无符号 32 位整数，将其保存在 `-0x14(%rbp)`。随后调用 `p1_compute`，将返回值与输入比较：

```
mov %r12,%rdi
call 0x402050 <p1_compute>
cmp %eax,-0x14(%rbp)
sete %al
```

因此，需要恢复 `p1_compute` 的计算过程。该函数先保存 challenge，再通过移位准备参数：

```
mov %rdi,%rdx
mov %rdi,%rax
shr $0x20,%rax
mov %rdx,%rsi
mov %eax,%edi
shr $0x10,%rsi
call 0x402030 <p1_step1>
```

记

$$h = \mathrm{u32}(C \gg 32), \qquad m = \mathrm{u32}(C \gg 16), \qquad l = \mathrm{u32}(C).$$

其中 m 对应原 challenge 的第 16 至 47 位。三个辅助函数的关键指令为：

```
# p1_step1
xor $0xa5de,%esi
lea (%rsi,%rdi,2),%eax
```

```
# p1_step0
lea (%rdi,%rdi,4),%eax
add $0xbf84,%esi
xor %esi,%eax

# p1_step2
lea 0xd2d7(%rsi),%eax
xor %edi,%eax
```

结合调用前的参数赋值，可知程序依次执行：

$$t_1 = \mathrm{step}_1(h, m) = (m \oplus \texttt{0xa5de}) + 2h,$$
$$t_0 = \mathrm{step}_0(l, h) = (5l) \oplus (h + \texttt{0xbf84}),$$
$$y = \mathrm{step}_2(t_0, t_1) = (t_1 + \texttt{0xd2d7}) \oplus t_0.$$

其中 t_1 暂存于 `ecx`，`p1_step0` 不修改该寄存器，因此可供最后一次调用使用。所有结果保留低 32 位，最终输出 y。

2.2 Phase 2：数组递推

`phase_02` 要求输入六个无符号 32 位整数，依次保存在 `rbp-0x40` 至 `rbp-0x2c`。程序将 challenge 保存在 `r13`，首先检查首项：

```
mov %r13,%rax
mov -0x40(%rbp),%esi
shr $0x2f,%rax
add %eax,%eax
and $0x7e,%eax
add $0x4,%eax
cmp %eax,%esi
jne 0x402115
```

`0x2f` 为十进制 47，因此：

$$a_0 = (2(C \gg 47)\ \&\ \texttt{0x7e}) + 4.$$

随后程序循环检查其余五项，关键指令为：

```
mov $0x1a,%ecx
mov $0x1,%edx
# loop:
mov %r13,%rax
shr %cl,%rax
and $0x3f,%eax
xor %edx,%eax
lea (%rax,%rsi,2),%eax
```

```
mov -0x40(%rbp,%rdx,4),%esi
cmp %eax,%esi
jne 0x402115
add $0x1,%rdx
add $0x4,%ecx
cmp $0x6,%rdx
jne 0x40217a
```

每轮开始时，edx 为下标 i，esi 保存前一项 a_{i-1}。移位量从 26 开始，每轮增加 4；程序提取 6 位，与下标异或，再加上前一项的两倍。因此：

$$a_i = 2a_{i-1} + [(\mathrm{u32}(C \gg (4i+22)) \,\&\, \texttt{0x3f}) \oplus i], \qquad i = 1, \ldots, 5.$$

2.3 Phase 3：分支与跳转表

输出为两个整数 (a, b)：

$$a = \mathrm{u32}(C) \,\&\, 7, \quad x = \mathrm{u32}(C \gg 3) \oplus \mathrm{u32}(C \gg 32), \quad b = T_a(x).$$

在 GDB 中执行：

```
x/8gx 0x405820
```

读取从 0x405820 开始的八个跳转目标地址，结合各目标位置的反汇编，得到以下对应关系。

索引	分支函数	$T_a(x)$
0	p3_case_3	$5x + \texttt{0x42e4b}$
1	p3_case_1	$(x \oplus \texttt{0x387}) + (x \gg 1)$
2	p3_case_7	$(x \oplus \texttt{0x5a62}) + (x \gg 1)$
3	p3_case_2	$3x + \texttt{0x79cb}$
4	p3_case_0	$(4x) \oplus \texttt{0xd2b2}$
5	p3_case_5	$5x + \texttt{0x152cf}$
6	p3_case_6	$(2x) \oplus \texttt{0xe113}$
7	p3_case_4	$5x + \texttt{0x55f}$

以两个分支为例：

```
# p3_case_2
lea 0x79cb(%rdi,%rdi,2),%eax
# p3_case_7
mov %edi,%eax
shr $1,%edi
xor $0x5a62,%eax
add %edi,%eax
```

第一条得到 $3x + \texttt{0x79cb}$。第二组中，edi 右移，eax 与常量异或，最后相加。实现时按表索引选择对应公式。

2.4 Phase 4：递归调用

入口要求一个整数，challenge 保存在 `r12`。调用前的关键指令为：

```
shr $0x20,%rax
shr $0x7,%rcx
and $0x7f,%edi
mov $0x7f,%edx
xor %eax,%ecx
call p4_walk
cmp %eax,-0x14(%rbp)
```

结合 `esi=0`，得到 $x = \mathrm{u32}(C)\ \&\ 127$、$k = \mathrm{u32}(C \gg 7) \oplus \mathrm{u32}(C \gg 32)$，目标值为 $F(x, 0, 127, k)$。

辅助函数将原始 x 放入 `eax`，用 `edi` 计算中点：

```
mov %edx,%edi
sub %esi,%edi
shr $1,%edi
add %esi,%edi
cmp %eax,%edi
je 0x402380
```

记 $m = low + ((high - low) \gg 1)$，递推式为：

$$F(x, low, high, k) = \begin{cases} (m + \texttt{0x94}) \oplus k, & x = m, \\ 3F(x, low, m-1, k) \oplus (m + k) \oplus \texttt{0x15}, & x < m, \\ 3F(x, m+1, high, k) \oplus (m + k) \oplus \texttt{0x1f}, & x > m. \end{cases}$$

`cmp %edi,%eax; jb` 实现无符号 $x < m$。当前层通过 `lea (%rdi,%rcx,1),%ebx` 保存 $m + k$，递归返回后用 `lea (%rax,%rax,2),%eax` 计算三倍结果，再异或保存值及左右分支常量。回溯必须使用当前层的中点。

每次递归缩小区间，且保持 $low \le x \le high$，最多 8 层调用命中中点；左分支计算 $m - 1$ 时不会下溢。

2.5 Phase 5：链表遍历

关键访存指令显示：

```
mov 0x405b60(,%rax,8),%rdx
mov 0x4(%rdx),%esi
add (%rdx),%eax
mov 0x8(%rdx),%rdx
```

节点占 16 字节：偏移 0、4 为 32 位 v, tag，偏移 8 为 64 位 next。读取 `0x405ba0` 起的 12 个节点，按地址顺序编号，并用索引保存连接关系：

节点索引	v（十六进制）	tag（十六进制）	next 索引
0	d2cd	a61	5
1	fe69	63d	0
2	e583	b52	10
3	b699	734	1
4	973b	953	8
5	fb86	46b	4
6	8527	275	2
7	39e8	14c	11
8	ecac	526	6
9	3ef2	313	3
10	2e6a	016	7
11	4233	87f	空

起点索引表为 $[9,3,1,0,5,4,8,6]$。令

$$i = \mathrm{u32}(C)\ \&\ 7,\quad q = \mathrm{u32}(C \gg 3)\ \&\ 3,\quad s_0 = \mathrm{u32}(C \gg 6) \oplus \mathrm{u32}(C \gg 32) \oplus \texttt{0x7a7a}.$$

从 `heads[i]` 开始，每个节点更新 $s \leftarrow 5(s+v) \oplus tag$。首节点先处理，之后沿 next 前进 q 次。因此 $q = 0,1,2,3$ 时分别处理 1、2、3、4 个节点。输出为最终 s 和最后处理节点的 tag。

例如起点 7 对应节点 6，$q = 3$ 时路径为 $6 \to 2 \to 10 \to 7$，最后节点的 $tag = \texttt{0x14c}$。合法路径最多前进 3 条边，不会访问空指针。

2.6 Phase 6：函数指针调用

入口将完整 challenge 传给 `p6_dispatch`，输出一个整数。两次间接调用及其间处理为：

```
and $0x38,%ebx
and $0x7,%eax
call *0x405e00(,%rax,8)
xor $0x83fb,%eax
mov %eax,%edi
call *0x405e00(%rbx)
add $0x6067,%eax
```

第一次用索引乘 8，第二次直接用已计算的字节偏移，故：

$$x = \mathrm{u32}(C \gg 6) \oplus \mathrm{u32}(C \gg 32),\quad i = \mathrm{u32}(C)\ \&\ 7,\quad j = \mathrm{u32}(C \gg 3)\ \&\ 7.$$

最终 $y = T_j(T_i(x) \oplus \texttt{0x83fb}) + \texttt{0x6067}$。

函数表 8 项实际对应 $[op_4, op_3, op_1, op_5, op_0, op_2, op_5, op_3]$，重复项必须保留。六个函数为：

$$
\begin{aligned}
op_0(x) &= \mathrm{ROR}_{32}(x, 1),\\
op_1(x) &= ((x \gg 5) \oplus x) + \texttt{0xef36},\\
op_2(x) &= (x \oplus \texttt{0xcc0b}) + 4x,\\
op_3(x) &= \mathrm{ROL}_{32}(x, 5) \oplus \texttt{0xc33a},\\
op_4(x) &= ((x \ll 7) + \texttt{0xb7c2}) \oplus 7x,\\
op_5(x) &= (x + \texttt{0x8c13}) \oplus (x \gg 5).
\end{aligned}
$$

其中 `rol $5,%eax` 与 `ror $1,%eax` 为 32 位循环移位，分别还原成 `(x << 5) | (x >> 27)` 和 `(x >> 1) | (x << 31)`。op_4 的另一条寄存器路径由 $8x - x$ 得到 $7x$；op_5 的右移作用于原始输入，不作用于加法结果。

3 Secret 隐藏关

3.1 门控长度与局部对象布局

`secret_gate` 的关键检查为：

```
call strlen
sub $0x24,%rax
cmp $0x7f,%rax
jbe 0x402770
# copy destination: -0xd0(%rbp)
call secret_copy
cmpl $0x4e45504f,-0xb0(%rbp)
lea -0xac(%rbp),%rdi
call secret_phase
```

这是无符号长度比较，接受 $36 \le \mathrm{strlen}(input) \le 163$；小于 36 时减法下溢，不能通过。`secret_copy` 逐字节复制，连末尾零字节一起写入，本身无长度检查；外层检查限制了本题可接受长度。

相对复制起点的偏移	内容	推理
0–31	32 字节填充	到标记前的距离
32–35	`OPEN`	小端常量 `0x4e45504f`
36 起	三整数后缀	传给 `secret_phase`

无需覆盖返回地址或构造机器代码，只需在指定位置写入标记并满足后缀校验。`x/s 0x403183` 显示格式串为 `%u %u %u`。

3.2 三整数计算

`secret_gate` 的第二、第三参数分别为 Honor challenge C 和 context D。令：

$$p = \mathrm{u32}(C) \oplus \mathrm{u32}(C \gg 32), \qquad q = \mathrm{u32}(D) \oplus \mathrm{u32}(D \gg 32),$$
$$a = \mathrm{u32}(D)\ \&\ 3, \qquad t = (a \cdot \texttt{0xbfeb}) \oplus \mathrm{u32}(D \gg 16).$$

`secret_mix` 的实际指令为：

```
mov %edi,%eax
xor $0xbfeb,%edi
shr $1,%eax
xor $0x3746,%eax
add %edi,%eax
ret
```

因此 $M(x) = ((x \gg 1) \oplus \texttt{0x3746}) + (x \oplus \texttt{0xbfeb})$。第二、第三整数分别为：

$$b = M(p \oplus q) \oplus t, \qquad c = b \oplus \big(q + a + \texttt{0x3746} + (p \gg 1)\big).$$

虽然 `esi,ecx,r8d,r9d` 均属调用者保存寄存器，但这份 `secret_mix` 实际仅修改 `edi,eax`，所以调用后仍可使用它们保留的 p, a, q, t。此结论来自实际指令，不是调用约定保证。

最终输出 32 个 `A`，接 `OPEN`，再直接接十进制的 `a b c`。此串长度为 41 至 59 字节，不含末尾零字节，满足门控长度；`snprintf` 返回值仍需检查，确保缓冲区能容纳字符串和零终止符。

4 测试及运行结果

```
Public tests: 60/60
```

图 1: 六个正式关卡的公开测试结果

```
Secret public tests: 10/10
```

图 2: Secret 关的公开测试结果

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。
```

图 3: 六个正式关卡在 seed 314159 下的通关结果。

```
Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

图 4: Secret 关在 seed 314159 下的通关结果。

各关卡公开列表之外的 seed 单关验证记录见报告末尾。注：由于不知道 Secret 关单关测试的代码指令，其在 seed 314159 下的验证结果见上图。

5 辅助工具使用说明

Codex 协助解释指令、求解代码编写，ChatGPT 协助整理报告。



图 5: 第 1 关在 seed 314159 下的单关验证。

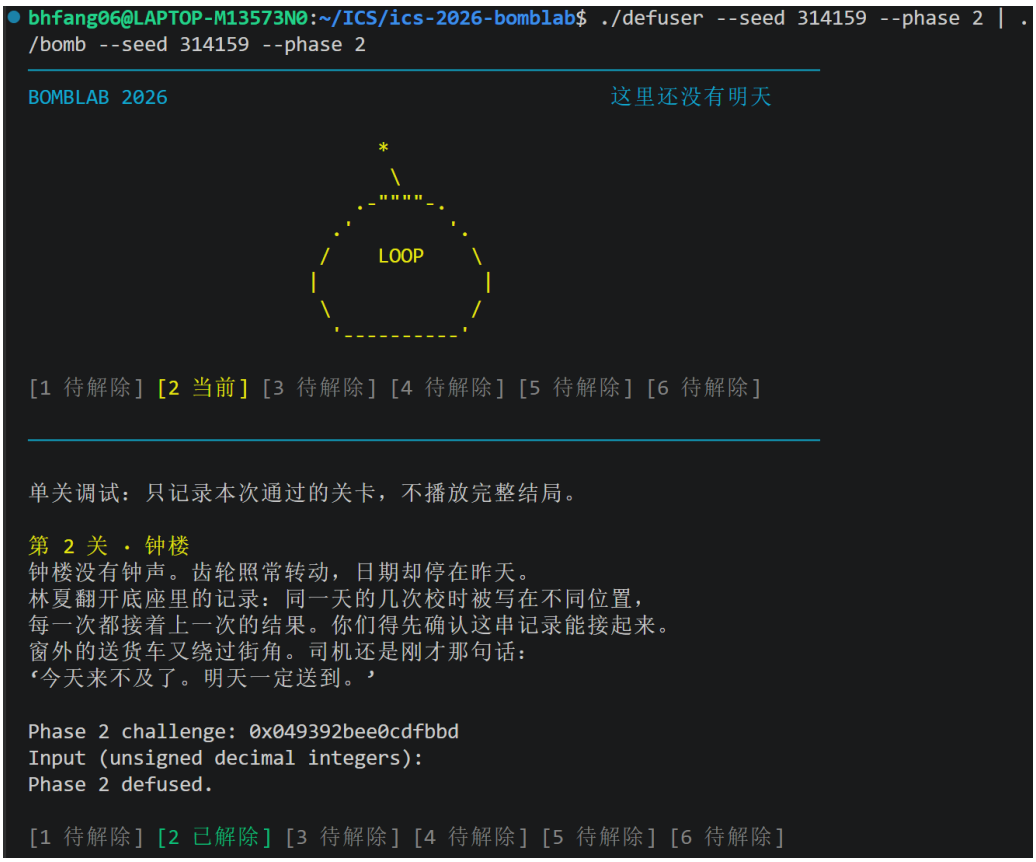


图 6: 第 2 关在 seed 314159 下的单关验证。



图 7: 第 3 关在 seed 314159 下的单关验证。



图 8: 第 4 关在 seed 314159 下的单关验证。



图 9: 第 5 关在 seed 314159 下的单关验证。



图 10: 第 6 关在 seed 314159 下的单关验证。